

GERENCIAMENTO DE LATÊNCIA EM APIS DE IA GENERATIVA: EFEITOS DE CACHE SEMÂNTICO, FILAS ASSÍNCRONAS E STREAMING SOBRE LATÊNCIA E VAZÃO

João Guilherme Aguilera Oliveira

Matheus Guida

Prof. Dr. João Emmanuel D'Alkmin Neves

FATEC Americana

joao.oliveira6@aluno.cps.sp.gov.br

Objetivos

Este trabalho tem como objetivo comparar, por meio de um cenário experimental controlado, três estratégias para o gerenciamento da latência em APIs de IA generativa: cache semântico, filas assíncronas e streaming de respostas. A comparação busca avaliar o comportamento dessas estratégias em um cenário experimental controlado, considerando métricas de latência e throughput. Além disso, pretende-se analisar os benefícios e as limitações de cada abordagem e suas implicações na responsividade de sistemas de IA generativa.

Métodos e Procedimentos

A comparação foi realizada por meio de um experimento controlado implementado em Python 3, avaliando três estratégias de gerenciamento de latência em APIs de IA generativa: cache semântico, fila assíncrona e *streaming*, utilizando uma condição síncrona como linha de base. A inferência foi simulada por um stub de latência controlada, mantendo as mesmas condições de processamento entre as estratégias. Foram utilizadas 35 requisições sintéticas, geradas de forma determinística, contendo perguntas únicas, equivalentes semânticos e casos lexicalmente semelhantes. No cache semântico, as requisições foram representadas por vetores e comparadas por similaridade de cosseno, cujo limiar foi

calibrado por varredura sobre a própria carga sintética (0,50 a 0,95), adotando-se o menor valor que preservasse precisão de 100% nessa varredura; um acerto foi considerado correto quando a pergunta consultada e a armazenada compartilhavam o mesmo rótulo semântico atribuído na geração da carga, e incorreto caso contrário. A fila assíncrona utilizou 8 *workers* concorrentes, enquanto o *streaming* permite avaliar separadamente o tempo até o primeiro retorno e o tempo de resposta completa. Foram avaliadas latência média, *throughput* e, conforme a estratégia, tempo até o primeiro retorno, taxa de acerto e precisão. Cada condição contou com 5 execuções de aquecimento e 30 repetições, com os tempos medidos por `time.perf_counter_ns()`.

Resultados

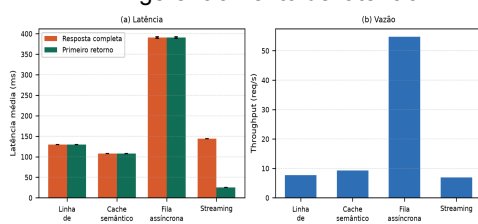
A Tabela 1 consolida as métricas obtidas. O cache semântico busca reutilizar respostas de consultas semanticamente semelhantes, reduzindo processamento redundante (BANG, 2023). Nesta implementação, reduziu a latência média em 17,0% frente à linha de base (129,83 ms para 107,72 ms) e elevou a vazão em 20,5%, com precisão de 100% no limiar calibrado (0,90, varredura de 0,50 a 0,95) e ocupação de apenas 3,0 KB de memória. A cobertura, porém, limitou-se a 17,5% das requisições: o limiar necessário para eliminar falsos positivos restringe o reaproveitamento, e

a calibração mostrou esse compromisso, limiares mais permissivos elevam a cobertura a 30%, mas introduzem respostas incorretas, limitação da representação lexical, na qual perguntas de sentido oposto podem apresentar similaridade superior à de paráfrases legítimas. A fila assíncrona produziu o maior ganho de vazão, de 7,70 para 54,72 req/s (7,1 vezes), ao custo de triplicar a latência individual (129,83 ms para 390,49 ms), decorrente de espera média em fila de 260,04 ms sob rajada, favorecendo a capacidade agregada de atendimento e não a requisição isolada. O streaming reduziu em 80,4% o tempo até o primeiro retorno (129,83 ms para 25,50 ms), ao custo de 11,0% a mais no tempo de resposta completa (144,07 ms) e queda de 9,9% na vazão, reflexo do custo de emissão incremental dos tokens (parte do acréscimo pode ainda decorrer da granularidade do temporizador do sistema operacional). O resultado ilustra a distinção entre latência efetiva e percebida: a estratégia onera levemente o processamento, mas antecipa substancialmente o instante em que o usuário vê o início da resposta. As três estratégias atuam, assim, sobre dimensões distintas do problema, sendo complementares e não excludentes.

Tabela 1: Desempenho das estratégias (média $\pm$ desvio padrão, N = 30 repetições)

Condição	Latência total (ms)	1º retorno (ms)	Vazão (req/s)
Linha de base	129,83 $\pm$ 0,05	129,83 $\pm$ 0,05	7,70
Cache semântico	107,72 $\pm$ 0,07	107,72 $\pm$ 0,07	9,28
Fila assíncrona	390,49 $\pm$ 1,89	390,49 $\pm$ 1,89	54,72
Streaming	144,07 $\pm$ 0,16	25,50 $\pm$ 0,01	6,94

Figura 1: Latência e vazão por estratégia de gerenciamento de latência.



Conclusões

Os resultados demonstram que as três estratégias apresentaram comportamentos distintos. O *cache* semântico reduziu a latência e aumentou a vazão, a fila assíncrona apresentou o maior ganho de *throughput*, porém com maior latência individual, e o *streaming* reduziu substancialmente o tempo até o primeiro retorno, possibilitando a antecipação do início da resposta, favorecendo a responsividade percebida. Dessa forma, a escolha deve considerar o objetivo do sistema: o *cache* semântico favorece o reaproveitamento de requisições, as filas assíncronas a capacidade de atendimento sob carga e o *streaming* a responsividade. As estratégias também podem ser combinadas para atuar sobre diferentes dimensões do desempenho de APIs de IA generativa.

Como limitação, os resultados foram obtidos em cenário experimental controlado, com condições específicas de carga e configuração, não permitindo generalização direta para ambientes de produção ou diferentes modelos de linguagem. Como continuidade, propõe-se avaliar combinações entre as estratégias, ampliar os cenários de carga e investigar seu comportamento em ambientes distribuídos e com diferentes modelos de IA generativa.

Os autores declaram não haver conflito de interesses. O autor João Guilherme Aguilera Oliveira atuou na estruturação e organização metodológica do trabalho, enquanto o autor Matheus Guida foi responsável pelo desenvolvimento do código e pela implementação do pipeline experimental. Todos os autores revisaram e aprovaram a versão final do manuscrito.

Referências bibliográficas

BANG, Fu. GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings. In: Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS), 2023, Singapura. p. 212-218.